3 Managing users

This chapter covers

- Describing a user with the `UserDetails` interface
- Using the `UserDetailsService` in the authentication flow
- Creating a custom implementation of the `UserDetailsService`
- Creating a custom implementation of `UserDetailsManager`
- Using the `JdbcUserDetailsManager` in the authentication flow

One of my colleagues from the university cooks pretty well. He's not a chef in a fancy restaurant, but he's quite passionate about cooking. One day, when sharing thoughts in a discussion, I asked him about how he manages to remember so many recipes. He told me that's easy. "You don't have to remember the whole recipe, but the way basic ingredients match with each other. It's like some real-world contracts that tell you what you can mix or should not mix. Then for each recipe, you only remember some tricks."

This analogy is similar to the way architectures work. With any robust framework, we use contracts to decouple the implementations of the framework from the application built on it. With Java, we use interfaces to define the contracts. A programmer is similar to a chef, knowing how the ingredients work together to choose just the right implementation. The programmer knows the framework's abstractions and uses those to integrate with it.

This chapter is about understanding in detail one of the fundamental roles you encountered in the first example we worked on in chapter 2—the `UserDetailsService`. Along with the `UserDetailsService`, we'll discuss the following interfaces (contracts):

- `UserDetails`, which describes the user for Spring Security.
- `GrantedAuthority`, which allows us to define actions that the user can execute.
- `UserDetailsManager`, which extends the `UserDetailsService` contract. Beyond the inherited behavior, it also describes actions such as creating a user and modifying or deleting a user's password.

From chapter 2, you already have an idea of the roles the `UserDetailsService` and the `PasswordEncoder` play in the authentication process. But we only discussed how to plug in an instance defined by you instead of using the default one configured by Spring Boot. We have more details to discuss, such as

- The implementations provided by Spring Security and how to use them
- How to define a custom implementation for contracts and when to do so
- Ways to implement interfaces that you find in real-world applications
- Best practices for using these interfaces

The plan is to start with how Spring Security understands the user definition. For this, we'll discuss the `UserDetails` and `GrantedAuthority` contracts. Next, we'll detail the `UserDetailsService` and how `UserDetailsManager` extends this contract. You'll apply implementations for these interfaces (e.g., `InMemoryUserDetailsManager`, `JdbcUserDetailsManager`, and `LdapUserDetailsManager`). When these implementations aren't a good fit for your system, you'll write a custom implementation.

3.1 Implementing authentication in Spring Security

In the previous chapter, we got started with Spring Security. In the first example, we discussed how Spring Boot specifies some defaults that define how a new application initially works. You have also learned how to override these defaults using various alternatives that we often find in apps. However, we only considered the surface of these so that you have an idea of what we'll be doing. In this chapter, as well as chapters 4 and 5, we'll discuss these interfaces in more detail, together with different implementations and where you might find them in real-world applications.

Figure 3.1 presents the authentication flow in Spring Security. This architecture is the backbone of the authentication process as implemented by Spring Security. It's important to understand it because you'll rely on it in any Spring Security implementation. You'll observe that we discuss parts of this architecture in almost all the chapters of this book. You'll see it so often that you'll probably learn it by heart, which is good. If you know this architecture, you're like a chef who knows their ingredients and can put together any recipe.

In figure 3.1, the shaded boxes represent the components that we start with: the `UserDetailsService` and `PasswordEncoder`. These two components focus on the part of the flow that I often refer to as "the user management part." In this chapter, the `UserDetailsService` and the `PasswordEncoder` are the components that deal directly

with user details and their credentials. We'll discuss the `PasswordEncoder` in detail in chapter 4.

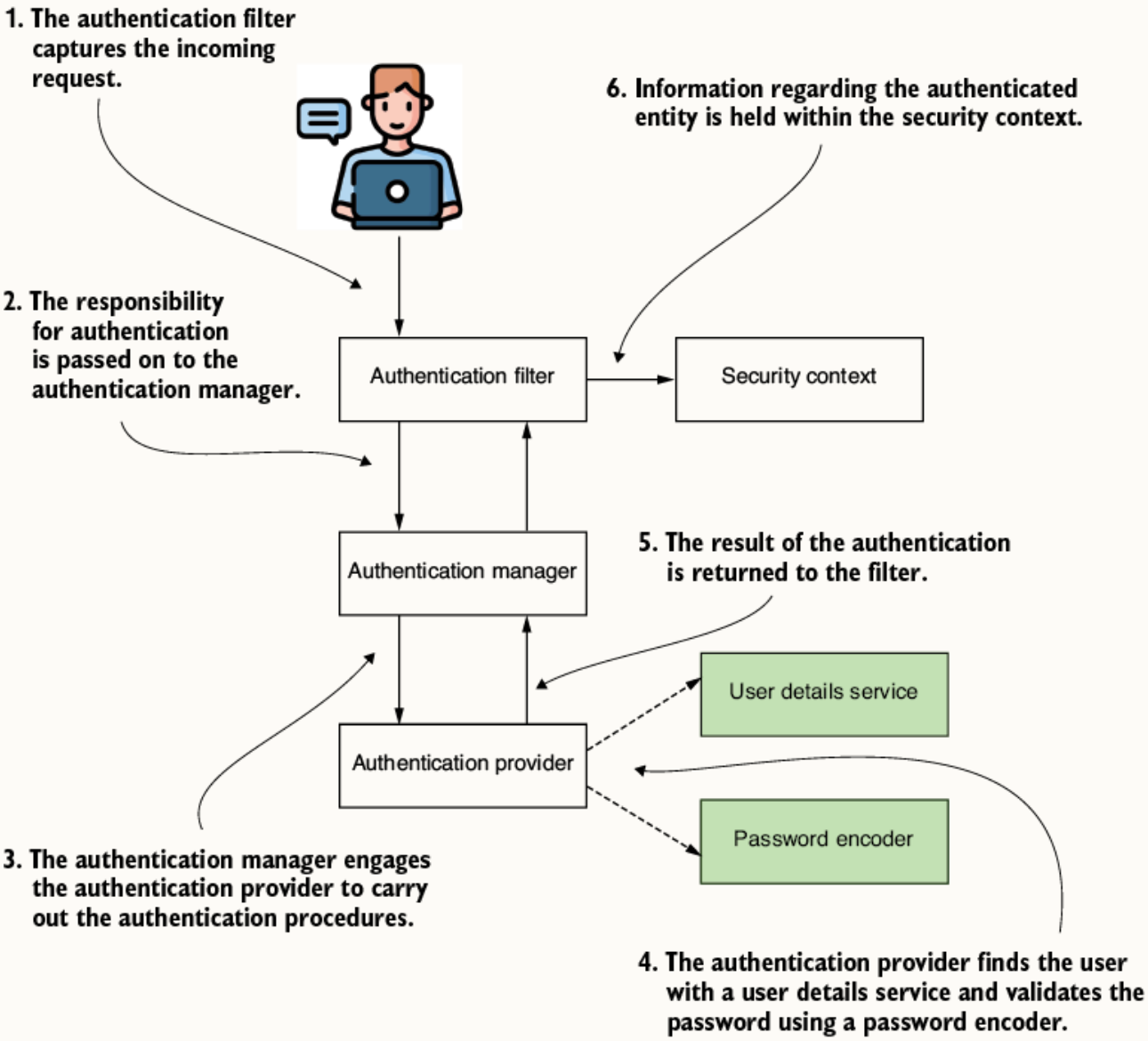


Figure 3.1 Spring Security's authentication flow. The AuthenticationFilter captures the incoming request and passes the task of authentication to the `AuthenticationManager`. The `AuthenticationManager`, in turn, utilizes an authentication provider to carry out the authentication process. For verifying the username and password, the `AuthenticationProvider` relies on a `UserDetailsService` and a `PasswordEncoder`.

As part of user management, we use the `UserDetailsService` and `UserDetailsManager` interfaces. The `UserDetailsService` is only responsible for retrieving the user by username. This action is the only one needed by the framework to complete authentication. The `UserDetailsManager` adds behavior that refers to adding, modifying, or deleting the user, which is a required functionality in most applications. The separation between the two contracts is an excellent example of the *interface segregation* principle. Separating the interfaces allows for better flexibility because the framework doesn't force you to implement behavior if your app doesn't need it. If the app only needs to authenticate the users, then implementing the `UserDetailsService` contract is enough to cover the desired functionality. To manage the users, `UserDetailsService` and the `UserDetailsManager` components need a way to represent them.

Spring Security offers the `UserDetails` contract, which you must implement to describe a user in the way the framework understands. As you'll learn in this chapter, in Spring Security, a user has a set of privileges, which are the actions the user is allowed to do. We'll work a lot with these privileges in chapters 7 through 12 when discussing authorization. But for now, Spring Security represents the actions that a user can do with the `GrantedAuthority` interface. We often call these *authorities*, and a user has one or more of them. In figure 3.2, you find a representation of the relationship between the components of the user management part of the authentication flow.

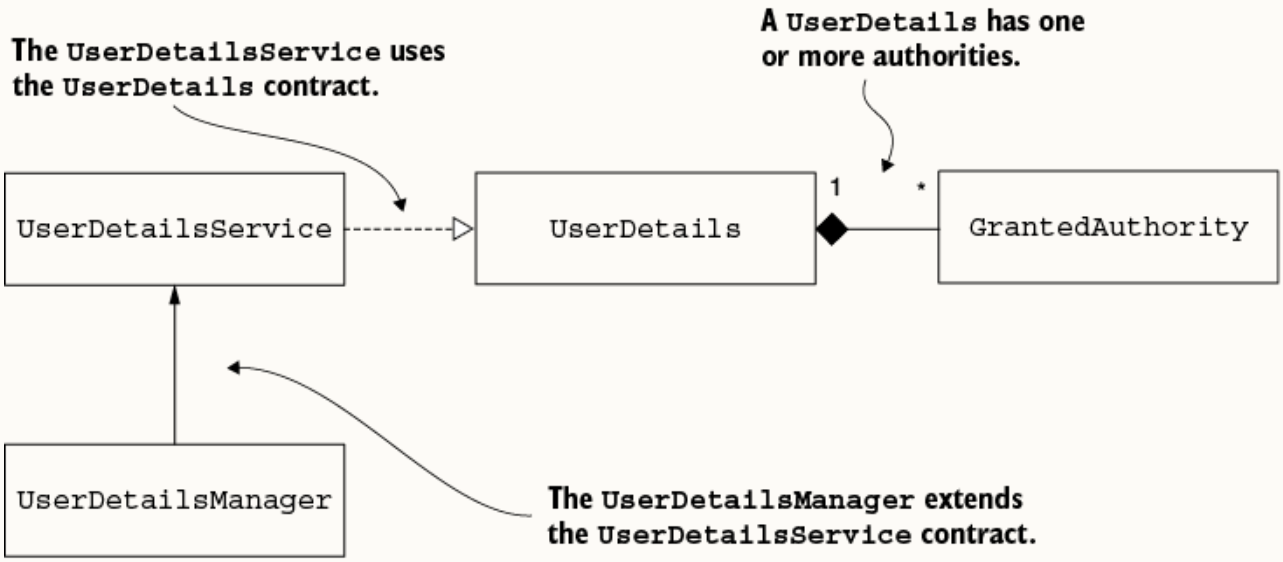


Figure 3.2 Dependencies between the components involved in user management. The `UserDetailsService` retrieves a user's details by searching for the user by name. The user is characterized by the `UserDetails` contract. Each user possesses one or more authorities, which are depicted by the `GrantedAuthority` interface. For incorporating operations such as create, delete, or modify password for a user, the `UserDetailsManager` contract, which expands on the `UserDetailsService`, is used to include these functionalities.

Understanding the links between these objects in the Spring Security architecture and ways to implement them gives you a wide range of options to choose from when working on applications. Any of these options could be the right puzzle piece in the app that you are working on, and you need to make your choice wisely. But to be able to choose, you first need to know what you can choose from.

3.2 Describing the user

In this section, you'll learn how to describe the users of your application so that Spring Security understands them. Learning how to represent users and make the framework aware of them is an essential step in building an authentication flow. Based on the user, the application makes a decision—whether a call to a certain functionality is allowed. To work with users, you first need to understand how to define the prototype of the user in your application. This section describes by example how to establish a blueprint for your users in a Spring Security application.

For Spring Security, a user definition should fulfill the `UserDetails` contract. The `UserDetails` contract represents the user as understood by Spring Security. The class of your application that describes the user must implement this interface, and in this way, the framework understands it.

3.2.1 Describing users with the UserDetails contract

In this section, you'll learn how to implement the `UserDetails` interface to describe the users in your application. We'll discuss the methods declared by the `UserDetails` contract to understand how and why we implement each of them. Let's start first by looking at the interface as presented in the following listing.

Listing 3.1 The `UserDetails` interface

```
public interface UserDetails extends Serializable {
  String getUsername();                                    ①
  String getPassword();
  Collection<? extends GrantedAuthority> getAuthorities();    ②
  boolean isAccountNonExpired();                           ③
  boolean isAccountNonLocked();
  boolean isCredentialsNonExpired();
  boolean isEnabled();
}
```

① These methods return the user credentials.

② Returns the actions that the app allows the user to do as a collection of GrantedAuthority instances

③ These four methods enable or disable the account for different reasons.

The `getUsername()` and `getPassword()` methods return, as you'd expect, the username and the password. The app uses these values in the authentication process, and these are the only details related to authentication from this contract. The other five methods all relate to authorizing the user to access the application's resources.

Generally, the app should allow a user to do some actions that are meaningful in the application's context. For example, the user should be able to read, write, or delete data. We say a user has or doesn't have the privilege to perform an action, and an authority represents the privilege a user has. We implement the `getAuthorities()` method to return the group of authorities granted to a user.

NOTE As you'll learn in chapter 6, Spring Security uses authorities to refer either to fine-grained privileges or to roles, which are groups of privileges. To make your reading more effortless, in this book, I refer to the fine-grained privileges as authorities.

Furthermore, as seen in the `UserDetails` contract, a user can

- Let the account expire
- Lock the account
- Let the credentials expire
- Disable the account

Suppose you choose to implement these user restrictions in your application's logic. In that case, you need to implement the methods `isAccountNonExpired()`, `isAccountNonLocked()`, `isCredentialsNonExpired()`, and `isEnabled()`, so that those needing to be enabled return true. Not all applications have accounts that expire or get locked with certain conditions. If you do not need to implement these functionalities in your application, you can simply make these four methods return true.

NOTE The names of the last four methods in the `UserDetails` interface may sound strange. One could argue that these are not wisely chosen in terms of clean coding and maintainability. For example, the name `isAccountNonExpired()` looks like a double negation, and at first sight, it might create confusion. But let's analyze all four method names with attention. These are named so that they all return false when the authorization should fail and true otherwise. This is the right approach because the human mind tends to associate the word "false" with negativity and the word "true" with positive scenarios.

3.2.2 Detailing on the GrantedAuthority contract

As you observed in the definition of the `UserDetails` interface in section 3.2.1, the actions granted for a user are called authorities. In chapters 7 through 12, we'll write authorization configurations based on these user authorities. Thus, it's essential to know how to define them.

The authorities represent what the user can do in your application. Without them, all users would be equal. While there are simple applications in which the users are equal, in most practical scenarios, an application defines multiple kinds of users. An application might have users who can only read specific information, while others can also modify the data. And you need to make your application differentiate between them, depending on the functional requirements of the application, which are the authorities a user needs. To describe the authorities in Spring Security, you use the `GrantedAuthority` interface.

Before we discuss implementing `UserDetails`, let's understand the `GrantedAuthority` interface. We use this interface in the definition of the user details. It represents a privilege granted to the user. A user must have at least one authority. Here's the implementation of the `GrantedAuthority` definition:

```
public interface GrantedAuthority extends Serializable {
   String getAuthority();
}
```

To create an authority, you only need to find a name for that privilege so you can refer to it later when writing the authorization rules. For example, a user can read the records managed by the application or delete them. You write the authorization rules based on the names you give to these actions.

In this chapter, we'll implement the `getAuthority()` method to return the authority's name as a `String`. The `GrantedAuthority` interface has only one abstract method, and in this book, you'll often find examples in which we use a lambda expression for its implementation. Another possibility is to use the `SimpleGrantedAuthority` class to create authority instances. The `SimpleGrantedAuthority` class offers a way to create immutable instances of the type `GrantedAuthority`. You provide the authority name when building the instance. In the next code snippet, you'll find two examples of implementing a `GrantedAuthority`. Here we make use of a lambda expression and then use the `SimpleGrantedAuthority` class:

```
GrantedAuthority g1 = () -> "READ";
GrantedAuthority g2 = new SimpleGrantedAuthority("READ");
```

3.2.3 Writing a minimal implementation of UserDetails

In this section, you'll write your first implementation of the `UserDetails` contract. We start with a basic implementation in which each method returns a static value. Then we change it to a version that you'll more likely find in a practical scenario, one that allows you to have multiple and different instances of users. Now that you know how to implement the `UserDetails` and `GrantedAuthority` interfaces, we can write the simplest user definition for an application.

With a class named `DummyUser`, let's implement a minimal description of a user, as in the following listing. I use this class mainly to demonstrate implementing the methods

for the `UserDetails` contract. Instances of this class always refer to only one user, `"bill"`, who has the password `"12345"` and an authority named `"READ"`.

Listing 3.2 The `DummyUser` class

```
public class DummyUser implements UserDetails {

  @Override
  public String getUsername() {
   return "bill";
  }

  @Override
  public String getPassword() {
   return "12345";
  }

  // Omitted code

}
```

The class in listing 3.2 implements the `UserDetails` interface and needs to implement all its methods. You will find here the implementation of `getUsername()` and `getPassword()`. In this example, these methods only return a fixed value for each of the properties.

Next, we add a definition for the list of authorities. The next listing shows the implementation of the `getAuthorities()` method. This method returns a collection with only one implementation of the `GrantedAuthority` interface.

Listing 3.3 Implementation of the `getAuthorities()` method

```
public class DummyUser implements UserDetails {

  // Omitted code

  @Override
  public Collection<? extends GrantedAuthority> getAuthorities() {
   return List.of(() -> "READ");
  }

  // Omitted code
```

```
}
```

Finally, you have to add an implementation for the last four methods of the `UserDetails` interface. For the `DummyUser` class, these always return true, meaning the user is forever active and usable. You can find the examples in the following listing.

Listing 3.4 Implementation of the last four `UserDetails` interface methods

```
public class DummyUser implements UserDetails {

  // Omitted code

  @Override
  public boolean isAccountNonExpired() {
    return true;
  }

  @Override
  public boolean isAccountNonLocked() {
    return true;
  }

  @Override
  public boolean isCredentialsNonExpired() {
    return true;
  }

  @Override
  public boolean isEnabled() {
    return true;
  }

  // Omitted code

}
```

Of course, this minimal implementation means that all instances of the class represent the same user. It's a good start to understanding the contract, but not something you would do in a real application. For a real application, you should create a class that you can use to generate instances that can represent different users. In this case, your defini-

tion would at least have the username and the password as attributes in the class, as shown in the next listing.

Listing 3.5 A more practical implementation of the `UserDetails` interface

```
public class SimpleUser implements UserDetails {

  private final String username;
  private final String password;

  public SimpleUser(String username, String password) {
    this.username = username;
    this.password = password;
  }

  @Override
  public String getUsername() {
    return this.username;
  }

  @Override
  public String getPassword() {
    return this.password;
  }

  // Omitted code

}
```

3.2.4 Using a builder to create instances of the UserDetails type

Some applications are simple and don't need a custom implementation of the `UserDetails` interface. In this section, we take a look at using a builder class provided by Spring Security to create simple user instances. Instead of declaring one more class in your application, you quickly obtain an instance representing your user with the `User` builder class.

The `User` class from the org.springframework.security.core.userdetails package is a simple way to build instances of the `UserDetails` type. Using this class, you can create immutable instances of `UserDetails`. You need to provide at least a username and a password, and the username shouldn't be an empty string. The following listing demonstrates how to use this builder. Building the user in this way, you don't need to have a custom implementation of the `UserDetails` contract.

Listing 3.6 Constructing a user with the `User` builder class

```
UserDetails u = User.withUsername("bill")
            .password("12345")
            .authorities("read", "write")
            .accountExpired(false)
            .disabled(true)
            .build();
```

With the previous listing as an example, let's dive deeper into the anatomy of the `User` builder class. The `User.withUsername(String username)` method returns an instance of the builder class `UserBuilder` nested in the `User` class. Another way to create the builder is by starting from another instance of `UserDetails`. In listing 3.7, the first line constructs a `UserBuilder`, starting with the username given as a string. Afterward, we demonstrate how to create a builder beginning with an already existing instance of `UserDetails`.

Listing 3.7 Creating the `User.UserBuilder` instance

```
User.UserBuilder builder1 = User.withUsername("bill");      ①

UserDetails u1 = builder1
            .password("12345")
            .authorities("read", "write")
            .passwordEncoder(p -> encode(p))          ②
            .accountExpired(false)
            .disabled(true)
            .build();                        ③

User.UserBuilder builder2 = User.withUserDetails(u);      ④

UserDetails u2 = builder2.build();
```

① Builds a user with their username

② The password encoder is only a function that does an encoding.

③ At the end of the build pipeline, calls the build() method

④ You can also build a user from an existing UserDetails instance.

You can see with any of the builders defined in listing 3.7 that it is possible to use the builder to obtain a user represented by the `UserDetails` contract. At the end of the build pipeline, you call the `build()` method. It applies the function defined to encode the password if you provide one, constructs the instance of `UserDetails`, and returns it.

NOTE Note that the password encoder is given here as a `Function<String, String>` and not in the form of the `PasswordEncoder` interface provided by Spring Security. This function's only responsibility is to transform a password in a given encoding. In the next section, we'll discuss in detail the `PasswordEncoder` contract from Spring Security that we used in chapter 2. We'll discuss the `PasswordEncoder` contract in more detail in chapter 4.

3.2.5 Combining multiple responsibilities related to the user

In the previous section, you learned how to implement the `UserDetails` interface. In real-world scenarios, it's often more complicated. In most cases, you find multiple responsibilities to which a user relates. And if you store users in a database, and then in the application, you would need a class to represent the persistence entity as well. Or if you retrieve users through a web service from another system, then you would probably need a data transfer object to represent the user instances. Assuming the first, a simple but also typical case, let's consider we have a table in an SQL database where we store the users. To make the example shorter, we give each user only one authority. The following listing shows the entity class that maps the table.

Listing 3.8 Defining the JPA `User` entity class

```
@Entity
public class User {

  @Id
  private Long id;
  private String username;
  private String password;
  private String authority;

  // Omitted getters and setters

}
```

If you make the same class also implement the Spring Security contract for user details, the class becomes more complicated. What do you think about how the code looks in the

next listing? In my opinion, it is a mess. I would get lost in it.

Listing 3.9 The `User` class has two responsibilities

```
@Entity
public class User implements UserDetails {

  @Id
  private int id;
  private String username;
  private String password;
  private String authority;

  @Override
  public String getUsername() {
    return this.username;
  }

  @Override
  public String getPassword() {
    return this.password;
  }

  public String getAuthority() {
    return this.authority;
  }

  @Override
  public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(() -> authority);
  }

  // Omitted code

}
```

The class contains JPA annotations, getters, and setters, of which both `getUsername()` and `getPassword()` override the methods in the `UserDetails` contract. It has a `getAuthority()` method that returns a `String`, as well as a `getAuthorities()` method that returns a `Collection`. The `getAuthority()` method is just a getter in the class, while `getAuthorities()` implements the method in the `UserDetails` interface. Things get even more complicated when adding relationships to other entities. Again, this code isn't friendly at all!

How can we write this code to be cleaner? The root of the muddy aspect of the previous code example is a mix of two responsibilities. While it's true that you need both in the application, in this case, nobody says that you have to put these into the same class. Let's try to separate those by defining a separate class called `SecurityUser`, which adapts the `User` class. As the next listing shows, the `SecurityUser` class implements the `UserDetails` contract and uses that to plug our user into the Spring Security architecture. The `User` class has only its JPA entity responsibility remaining.

Listing 3.10 Implementing the `User` class only as a JPA entity

```
@Entity
public class User {

  @Id
  private int id;
  private String username;
  private String password;
  private String authority;

  // Omitted getters and setters

}
```

The `User` class in listing 3.10 has only its JPA entity responsibility remaining, and thus, it becomes more readable. If you read this code, you can now focus exclusively on details related to persistence, which are not important from the Spring Security perspective. In the next listing, we implement the `SecurityUser` class to wrap the `User` entity.

Listing 3.11 The `SecurityUser` class implementing the `UserDetails` contract

```
public class SecurityUser implements UserDetails {

  private final User user;

  public SecurityUser(User user) {
    this.user = user;
  }

  @Override
  public String getUsername() {
    return user.getUsername();
  }
```

```
    @Override
    public String getPassword() {
      return user.getPassword();
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
      return List.of(() -> user.getAuthority());
    }

    // Omitted code

  }
```

As you can observe, we use the `SecurityUser` class only to map the user details in the system to the `UserDetails` contract understood by Spring Security. To mark the fact that the `SecurityUser` makes no sense without a `User` entity, we make the field final. You have to provide the user through the constructor. The `SecurityUser` class adapts the `User` entity class and adds the needed code related to the Spring Security contract without mixing the code into a JPA entity, thereby implementing multiple different tasks.

NOTE You can find different approaches to separate the two responsibilities. I don't want to say that the approach I present in this section is the best or the only one. Usually, the way you choose to implement the class design varies a lot from one case to another. But the main idea is the same: avoid mixing responsibilities and try to write your code to be as decoupled as possible to increase the maintainability of your app.

3.3 Instructing Spring Security on how to manage users

In the previous section, you implemented the `UserDetails` contract to describe users so that Spring Security understands them. But how does Spring Security manage users? Where are they taken from when comparing credentials, and how do you add new users or change existing ones? In chapter 2, you learned that the framework defines a specific component to which the authentication process delegates user management: the `UserDetailsService` instance. We even defined a `UserDetailsService` to override the default implementation provided by Spring Boot.

In this section, we experiment with various ways of implementing the `UserDetailsService` class. You'll understand how user management works by implementing the responsibility described by the `UserDetailsService` contract in our example.

After that, you'll find out how the `UserDetailsManager` interface adds more behavior to the contract defined by the `UserDetailsService`. At the end of this section, we'll use the provided implementations of the `UserDetailsManager` interface offered by Spring Security. We'll write an example project where we'll use one of the best-known implementations provided by Spring Security, the `JdbcUserDetailsManager` class. After learning this, you'll know how to tell Spring Security where to find users, which is essential in the authentication flow.

3.3.1 Understanding the UserDetailsService contract

In this section, you'll learn about the `UserDetailsService` interface definition. Before understanding how and why to implement it, you must first understand the contract. It is time to go into more detail on `UserDetailsService` and how to work with implementations of this component. The `UserDetailsService` interface contains only one method, as follows:

```
public interface UserDetailsService {

  UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException;
}
```

The authentication implementation calls the `loadUserByUsername(String username)` method to obtain the details of a user with a given username (figure 3.3). The username is, of course, considered unique. The user returned by this method is an implementation of the `UserDetails` contract. If the username doesn't exist, the method throws a `UsernameNotFoundException`.

NOTE The `UsernameNotFoundException` is a `RuntimeException`. The `throws` clause in the `UserDetailsService` interface is only for documentation purposes. The `UsernameNotFoundException` inherits directly from the type `AuthenticationException`, which is the parent of all the exceptions related to the process of authentication. `AuthenticationException` further inherits the `RuntimeException` class.

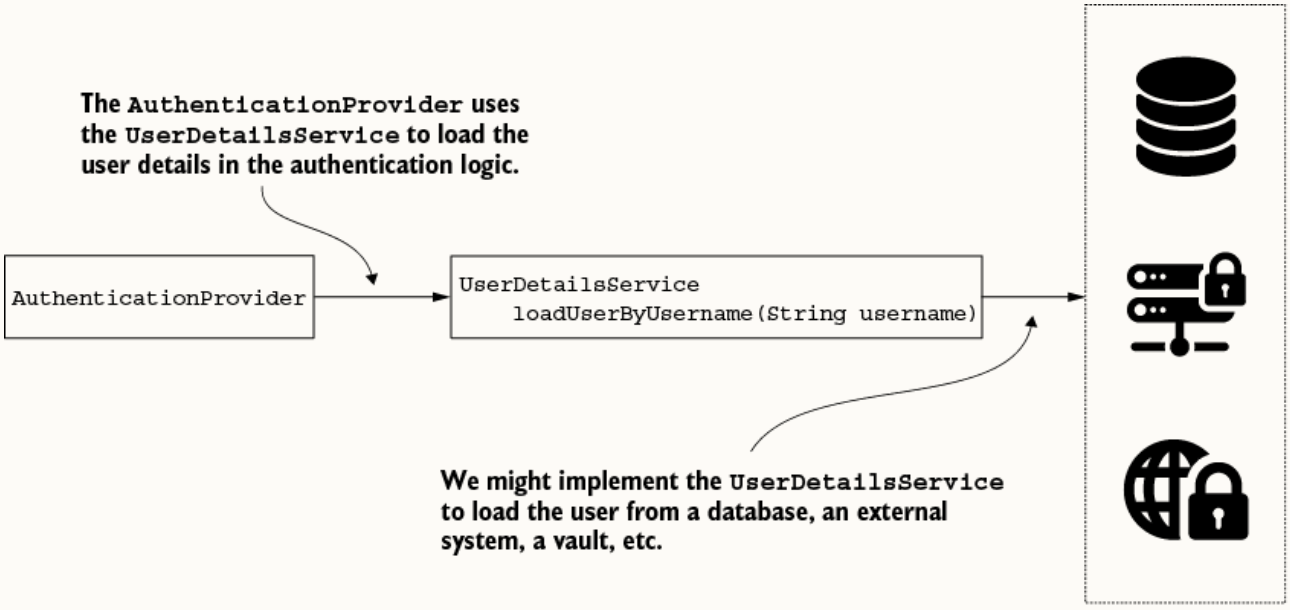


Figure 3.3 The `AuthenticationProvider` is the element responsible for executing the authentication process and utilizes the `UserDetailsService` to gather user details. It invokes the `loadUserByUsername(String username)` method to locate the user based on their username.

3.3.2 Implementing the UserDetailsService contract

In this section, we work on a practical example to demonstrate the implementation of the `UserDetailsService`. Your application manages details about credentials and other user aspects. It could be that these are stored in a database or handled by another system that you access through a web service or by other means (figure 3.3). Regardless of how this happens in your system, the only thing Spring Security needs from you is an implementation to retrieve the user by username.

In the next example, we write a `UserDetailsService` that has an in-memory list of users. In chapter 2, you used a provided implementation that does the same thing, the `InMemoryUserDetailsManager`. Because you are already familiar with how this implementation works, I have chosen a similar functionality, but this time to implement on our own. We provide a list of users when we create an instance of our `UserDetailsService` class. You can find this example in the project ssia-ch3-ex1. In the package named model, we define the `UserDetails` as presented by the following listing.

Listing 3.12 The implementation of the `UserDetails` interface

```
public class User implements UserDetails {

  private final String username;                ①
  private final String password;
  private final String authority;               ②

  public User(String username, String password, String authority) {
```

```
    this.username = username;
    this.password = password;
    this.authority = authority;
  }

  @Override
  public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(() -> authority);              ③
  }

  @Override
  public String getPassword() {
    return password;
  }

  @Override
  public String getUsername() {
    return username;
  }

  @Override
  public boolean isAccountNonExpired() {          ④
    return true;
  }

  @Override
  public boolean isAccountNonLocked() {
    return true;
  }

  @Override
  public boolean isCredentialsNonExpired() {
    return true;
  }

  @Override
  public boolean isEnabled() {
    return true;
  }
}
```

① The User class is immutable. You give the values for the three attributes when you build the instance, and these values cannot be changed afterward.

② To make the example simple, a user has only one authority.

③ Returns a list containing only the GrantedAuthority object with the name provided when you built the instance

④ The account does not expire or get locked.

In the package named services, we create a class called `InMemoryUserDetailsService`. The following listing shows how we implement this class.

Listing 3.13 The implementation of the `UserDetailsService` interface

```
public class InMemoryUserDetailsService implements UserDetailsService {

  private final List<UserDetails> users;                    ①

  public InMemoryUserDetailsService(List<UserDetails> users) {
    this.users = users;
  }

  @Override
  public UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException {

    return users.stream()
      .filter(
        u -> u.getUsername().equals(username)
      )
      .findFirst()                                          ②
      .orElseThrow(                                         ③
        () -> new UsernameNotFoundException("User not found")
      );
  }
}
```

① UserDetailsService manages the list of users in-memory.

② If there is such a user, returns it

③ If a user with this username does not exist, throws an exception

The `loadUserByUsername(String username)` method searches the list of users for the given username and returns the desired `UserDetails` instance. If there is no instance with that username, it throws a `UsernameNotFoundException`. We can now use this im-

plementation as our `UserDetailsService`. The next listing shows how we add it as a bean in the configuration class and register one user within it.

Listing 3.14 `UserDetailsService` registered as a bean in the configuration class

```
@Configuration
public class ProjectConfig {

  @Bean
  public UserDetailsService userDetailsService() {
    UserDetails u = new User("john", "12345", "read");
    List<UserDetails> users = List.of(u);
    return new InMemoryUserDetailsService(users);
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

Finally, we create a simple endpoint and test the implementation. The following listing defines the endpoint.

Listing 3.15 The definition of the endpoint used for testing the implementation

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
  }
}
```

When calling the endpoint using cURL, we observe that for user `john` with password `12345`, we get back an HTTP 200 OK. If we use something else, the application returns 401 Unauthorized:

```
curl -u john:12345 http://localhost:8080/hello
```

The response body is

```
Hello!
```

3.3.3 Implementing the UserDetailsManager contract

In this section, we discuss using and implementing the `UserDetailsManager` interface. This interface extends and adds more methods to the `UserDetailsService` contract. Spring Security needs the `UserDetailsService` contract to do the authentication. But generally, in applications, there is also a need for managing users. Most of the time, an app should be able to add new users or delete existing ones. In this case, we implement a more particular interface defined by Spring Security, `UserDetailsManager`. It extends `UserDetailsService` and adds more operations that we need to implement:

```
public interface UserDetailsManager extends UserDetailsService {
  void createUser(UserDetails user);
  void updateUser(UserDetails user);
  void deleteUser(String username);
  void changePassword(String oldPassword, String newPassword);
  boolean userExists(String username);
}
```

The `InMemoryUserDetailsManager` object that we used in chapter 2 is actually a `UserDetailsManager`. At that time, we only considered its `UserDetailsService` characteristics. Project ssia-ch3-ex2 accompanies the example in this section.

Using a JdbcUserDetailsManager for user management

Besides the `InMemoryUserDetailsManager`, we often use another `UserDetailManager` implementation, `JdbcUserDetailsManager`. The `JdbcUserDetailsManager` class manages users in an SQL database. It connects to the database directly through JDBC. This way, the `JdbcUserDetailsManager` is independent of any other framework or specification related to database connectivity.

To understand how the `JdbcUserDetailsManager` works, it's best if you put it into action with an example. In the following example, you implement an application that manages the users in a MySQL database using the `JdbcUserDetailsManager`. Figure 3.4 provides an overview of the place the `JdbcUserDetailsManager` implementation takes in the authentication flow.

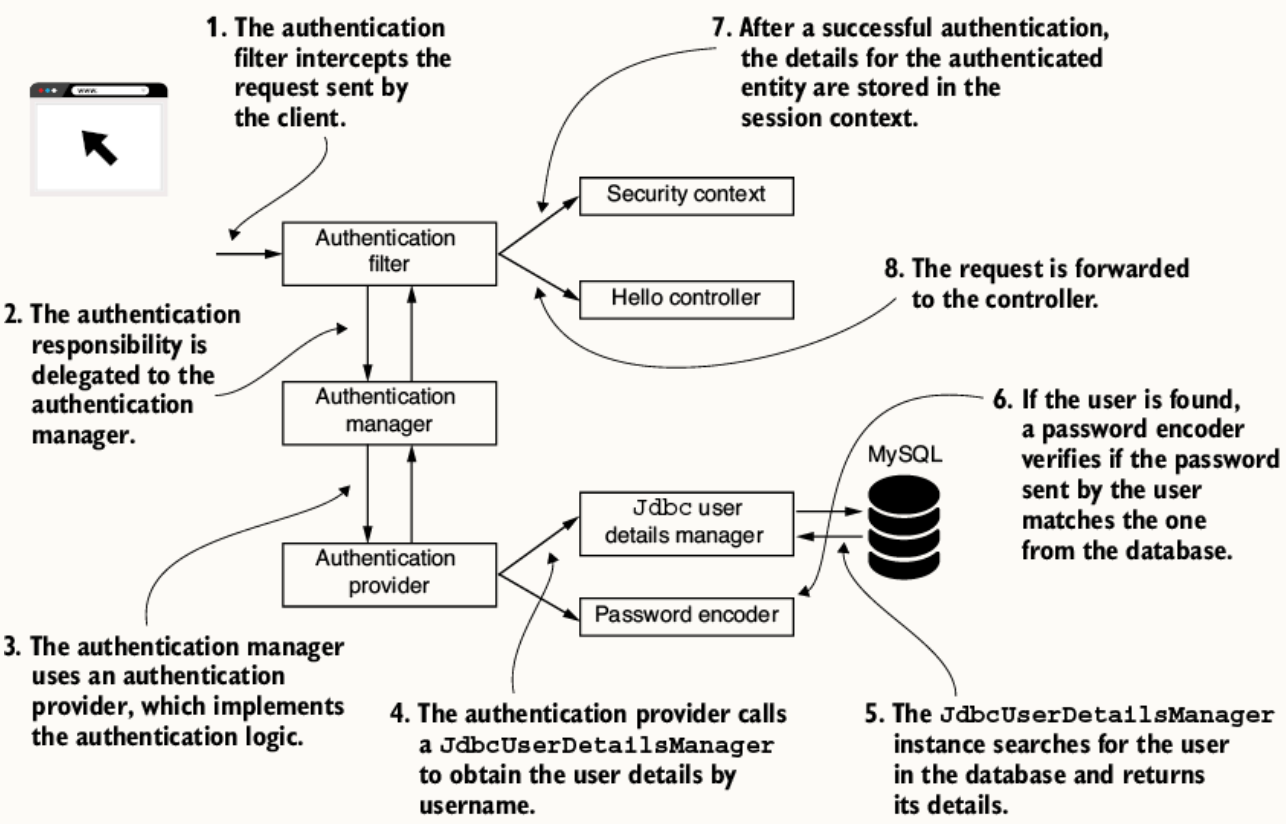


Figure 3.4 The Spring Security authentication flow. Here we use a `JdbcUserDetailsManager` as our `UserDetailsService` component. The `JdbcUserDetailsManager` uses a database to manage users.

You'll start working on our demo application that uses the `JdbcUserDetailsManager` by creating a database and two tables. In our case, we name the database `spring`, and we name one of the tables `users` and the other `authorities`. These names are the default table names known by the `JdbcUserDetailsManager`. As you'll learn at the end of this section, the `JdbcUserDetailsManager` implementation is flexible and lets you override these default names if you want to do so. The purpose of the users table is to keep user records. The `JdbcUserDetailsManager` implementation expects three columns in the `users` table—a username, a password, and enabled—which you can use to deactivate the user.

You can choose to create the database and its structure yourself by using either the command-line tool for your database management system (DBMS) or a client application. For example, for MySQL, you can choose to use MySQL Workbench to do this. But the easiest thing to do would be to let Spring Boot itself run the scripts for you. To do this, just add two more files to your project in the resources folder: schema.sql and data.sql. In the schema.sql file, you add the queries related to the database structure, such as creating, altering, or dropping tables. In the data.sql file, you add the queries that work with the data inside the tables, such as `INSERT`, `UPDATE`, or `DELETE`. Spring Boot automatically runs these files for you when you start the application. A simpler solution for building examples that need databases is using an H2 in-memory database. This way, you don't need to install a separate DBMS solution.

NOTE If you prefer, you could go with H2 as well (as I do in the ssia-ch3-ex2 project) when developing the applications presented in this book. But in most cases, I chose to implement the examples with an external DBMS to make it clear that it's an external component of the system and avoid confusion in this way.

You use the code in the next listing to create the `users` table with a MySQL server. You can add this script to the schema.sql file in your Spring Boot project.

Listing 3.16 The SQL query for creating the `users` table

```
CREATE TABLE IF NOT EXISTS `spring`.`users` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `username` VARCHAR(45) NOT NULL,
  `password` VARCHAR(45) NOT NULL,
  `enabled` INT NOT NULL,
  PRIMARY KEY (`id`));
```

The `authorities` table stores authorities per user. Each record stores a username and an authority granted for the user with that username.

Listing 3.17 The SQL query for creating the `authorities` table

```
CREATE TABLE IF NOT EXISTS `spring`.`authorities` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `username` VARCHAR(45) NOT NULL,
  `authority` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`id`));
```

NOTE For simplicity and to allow you focus on the Spring Security configurations we discuss, in the examples provided with this book, I skip the definitions of indexes or foreign keys.

To make sure you have a user for testing, insert a record in each of the tables. You can add these queries in the data.sql file in the resources folder of the Spring Boot project:

```
INSERT INTO `spring`.`authorities`
(username, authority)
VALUES
('john', 'write');

INSERT INTO `spring`.`users`
```

```
(username, password, enabled)
VALUES
('john', '12345', '1');
```

For your project, you need to add at least the dependencies stated in the following listing. Check your pom.xml file to make sure you added these dependencies.

Listing 3.18 Dependencies needed to develop the example project

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
 <groupId>com.h2database</groupId>
 <artifactId>h2</artifactId>
</dependency>
```

NOTE In your examples, you can use any SQL database technology as long as you add the correct JDBC driver to the dependencies.

Remember, you need to add the JDBC driver according to the database technology you use. For example, if you'd use MySQL, you need to add the MySQL driver dependency as presented in the next snippet:

```
<dependency>
 <groupId>mysql</groupId>
 <artifactId>mysql-connector-java</artifactId>
 <scope>runtime</scope>
</dependency>
```

You can configure a data source in the application.properties file of the project or as a separate bean. If you choose to use the application.properties file, you need to add the

following lines to that file:

```
spring.datasource.url=jdbc:h2:mem:ssia
spring.datasource.username=sa
spring.datasource.password=
spring.sql.init.mode=always
```

In the configuration class of the project, you define the `UserDetailsService` and the `PasswordEncoder`. The `JdbcUserDetailsManager` needs the `DataSource` to connect to the database. The data source can be autowired through a parameter of the method (as presented in the next listing) or through an attribute of the class.

Listing 3.19 Registering the `JdbcUserDetailsManager` in the configuration class

```
@Configuration
public class ProjectConfig {

  @Bean
  public UserDetailsService userDetailsService(DataSource dataSource) {
    return new JdbcUserDetailsManager(dataSource);
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

To access any endpoint of the application, you now need to use HTTP Basic authentication with one of the users stored in the database. To prove this, we create a new endpoint, as shown in the following listing, and then call it with cURL.

Listing 3.20 The test endpoint to check the implementation

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
```

```
  }
}
```

In the next code snippet, you find the result when calling the endpoint with the correct username and password:

```
curl -u john:12345 http://localhost:8080/hello
```

The response to the call is

```
Hello!
```

The `JdbcUserDetailsManager` also allows you to configure the queries used. In the previous example, we made sure we used the exact names for the tables and columns, as the `JdbcUserDetailsManager` implementation expects those. But it could be that these names are not the best choice for your application. The next listing shows how to override the queries for the `JdbcUserDetailsManager`.

Listing 3.21 Changing `JdbcUserDetailsManager`'s queries to find the user

```
@Bean
public UserDetailsService userDetailsService(DataSource dataSource) {
  String usersByUsernameQuery =
    "select username, password, enabled from users where username = ?";
  String authsByUserQuery =
    "select username, authority from spring.authorities where username = ?";

  var userDetailsManager = new JdbcUserDetailsManager(dataSource);
  userDetailsManager.setUsersByUsernameQuery(usersByUsernameQuery);
  userDetailsManager.setAuthoritiesByUsernameQuery(authsByUserQuery);
  return userDetailsManager;
}
```

In the same way, we can change all the queries used by the `JdbcUserDetailsManager` implementation.

EXERCISE Write a similar application for which you name the tables and the columns differently in the database. Override the queries for the `JdbcUserDetailsManager` im-

plementation (e.g., the authentication works with a new table structure). The project ssia-ch3-ex2 features a possible solution.

Using an LdapUserDetailsManager for user management

Spring Security also offers an implementation of `UserDetailsManager` for LDAP. Even if it is less popular than the `JdbcUserDetailsManager`, you can count on it if you need to integrate with an LDAP system for user management. In the project ssia-ch3-ex3, you can find a simple demonstration of using the `LdapUserDetailsManager`. Because I can't use a real LDAP server for this demonstration, I have set up an embedded one in my Spring Boot application. To set up the embedded LDAP server, I defined a simple LDAP Data Interchange Format (LDIF) file. The following listing shows the contents of my LDIF file.

Listing 3.22 The definition of the LDIF file

```
dn: dc=springframework,dc=org                    ①
objectclass: top
objectclass: domain
objectclass: extensibleObject
dc: springframework

dn: ou=groups,dc=springframework,dc=org          ②
objectclass: top
objectclass: organizationalUnit
ou: groups

dn: uid=john,ou=groups,dc=springframework,dc=org ③
objectclass: top
objectclass: person
objectclass: organizationalPerson
objectclass: inetOrgPerson
cn: John
sn: John
uid: john
userPassword: 12345
```

① Defines the base entity

② Defines a group entity

③ Defines a user

In the LDIF file, I add only one user for which we need to test the app's behavior at the end of this example. We can add the LDIF file directly to the resources folder. This way, it's automatically in the `classpath`, so we can easily refer to it later. I named the LDIF file server.ldif. To work with LDAP and to allow Spring Boot to start an embedded LDAP server, you need to add pom.xml to the dependencies:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-ldap</artifactId>
</dependency>
<dependency>
  <groupId>com.unboundid</groupId>
  <artifactId>unboundid-ldapsdk</artifactId>
</dependency>
```

In the application.properties file, you also need to add the configurations for the embedded LDAP server, as presented in the following code snippet. The values the app needs to boot the embedded LDAP server include the location of the LDIF file, a port for the LDAP server, and the base domain component (DN) label values:

```
spring.ldap.embedded.ldif=classpath:server.ldif
spring.ldap.embedded.base-dn=dc=springframework,dc=org
spring.ldap.embedded.port=33389
```

Once you have an LDAP server for authentication, you can configure your application to use it. The next listing shows how to configure the `LdapUserDetailsManager` to enable your app to authenticate users through the LDAP server.

Listing 3.23 The definition of the `LdapUserDetailsManager` in the configuration file

```
@Configuration
public class ProjectConfig {

  @Bean                                          ①
  public UserDetailsService userDetailsService() {
    var cs = new DefaultSpringSecurityContextSource(
"ldap://127.0.0.1:33389/dc=springframework,dc=org");    ②
    cs.afterPropertiesSet();

    var manager = new LdapUserDetailsManager(cs);       ③
```

```
    manager.setUsernameMapper(
new DefaultLdapUsernameToDnMapper("ou=groups", "uid")); ④

    manager.setGroupSearchBase("ou=groups");          ⑤

    return manager;
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }
}
```

① Adds a UserDetailsService implementation to the Spring context

② Creates a context source to specify the address of the LDAP server

③ Creates the LdapUserDetailsManager instance

④ Sets a username mapper to instruct the LdapUserDetailsManager on how to search for users

⑤ Sets the group search base that the app needs to search for users

Let's also create a simple endpoint to test the security configuration. I added a controller class, as presented in the next code snippet:

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String hello() {
    return "Hello!";
  }
}
```

Now start the app, and call the `/hello` endpoint. You need to authenticate with user `john` if you want the app to allow you to call the endpoint. The next code snippet shows you the result of calling the endpoint with cURL:

```
curl -u john:12345 http://localhost:8080/hello
```

The response to the call is

```
Hello!
```

Summary

- The `UserDetails` interface is the contract you use to describe a user in Spring Security.
- The `UserDetailsService` interface is the contract that Spring Security expects you to implement in the authentication architecture to describe the way the application obtains user details.
- The `UserDetailsManager` interface extends `UserDetailsService` and adds the behavior related to creating, changing, or deleting a user.
- Spring Security provides a few implementations of the `UserDetailsManager` contract. Among these are `InMemoryUserDetailsManager`, `JdbcUserDetailsManager`, and `LdapUserDetailsManager`.
- The `JdbcUserDetailsManager` class has the advantage of directly using JDBC and does not lock the application in to other frameworks.

Previous chapter
‹ Part 2. Configuring authentication

Next chapter
4 Managing passwords ›